

딥러닝 기반 실시간 투구 전략 최적화 시스템

— 데이터 누수의 발견과 파이프라인 재설계 —

개정판

22013442 황일준

요약

본 연구는 MLB Statcast 투구 단위 데이터를 이용해 투구 결과를 예측하고, 주어진 상황에서 기대 실점이 가장 낮은 구종과 코스를 제시하는 시스템을 구축한다. 초기 버전은 검증 정확도 67.52%를 보고하였으나, 본 개정 과정에서 그 수치가 **미래 정보 누수로 만들어진 값**임을 확인하였다. Statcast 원본이 역시간순으로 제공된다는 사실을 놓쳐, 결과적으로 나중에 던진 공들로 먼저 던진 공의 결과를 맞추는 문제를 학습하고 있었다.

이에 데이터 구성부터 평가 방법까지 전면 재설계하였다. 시퀀스를 시간순으로 정렬하고 타석 내부로 제한하였으며, 분할을 시즌 단위로 바꾸고, 불균형 이중 보정을 제거하였다. 2024 시즌 40 에폭 학습, 2025 시즌 712,516 구 홀드아웃 평가 결과 log-loss 1.1259, macro AUC 0.8410, ECE 0.0084 를 얻었다.

어블레이션을 통해 맥락 스킵 연결이 성능에 기여함을 확인하였으나, 동시에 **투구 시퀀스 자체의 기여는 예상보다 작다**는 사실도 함께 확인하였다. 이는 초기 연구가 전제한 '투구 배합의 시계열 맥락이 결정적'이라는 가정을 이 데이터가 강하게 지지하지 않음을 뜻한다.

1. 서론

현대 야구에서 투수와 타자의 승부는 볼카운트, 주자 상황, 직전 투구의 배합 등 매 순간 변화하는 맥락 속에서 이루어진다. 타율·출루율·장타율과 같은 누적 지표는 선수의 전반적 가치를 요약하는 데 유용하지만, 특정 시점의 의사결정을 돕지는 못한다. 투수와 포수가 실제로 필요로 하는 정보는 '지금 이 상황에서 어떤 공을 어디에 던져야 하는가'이며, 이는 결과가 아니라 **선택**에 관한 질문이다.

본 연구는 이 질문에 답하기 위해 투구 단위 결과 예측 모델을 학습하고, 그 확률 분포를 기대 실점으로 환산하여 후보 투구를 순위화하는 시스템을 구축한다. 다만 본 보고서의 서술 순서는

통상적인 연구 보고서와 다르다. 초기 구현이 산출한 수치가 유효하지 않음을 발견한 것이 본
개정의 출발점이므로, **무엇이 잘못되었는지를 먼저 규명한 뒤** 그에 대응하는 설계를 제시한다.

본 연구의 기여는 다음과 같다.

- Statcast 데이터를 이용한 시퀀스 모델링에서 발생하기 쉬운 **시간 역순 누수를 구체적으로 규명**하고, 그것이 성능 수치를 어떻게 부풀리는지 정량적으로 보였다.
- 타석 단위 시퀀스 구성, 시즌 단위 분할, 학습 구간 한정 정규화를 통해 **재현 가능하고 누수 없는 파이프라인**을 재구축하였다.
- 임의의 가중치에 의존하던 추천 점수를 Statcast 기대 득점 변화량에 근거한 **카운트 조건부 기대 실점**으로 대체하였다.
- 한 번에 한 요소만 변경하는 어블레이션으로 각 구성요소의 기여를 분리하고, **시퀀스의 기여가 작다는 부정적 결과를 포함해** 보고하였다.

2. 관련 연구

투구 결과 예측에는 오랫동안 트리 기반 모델이 사용되었다. Sidle 와 Tran 은 랜덤 포레스트로 구종을 예측하며 타자 유리 카운트에서 예측 가능성이 높아짐을 보였고, Swartz 등은 투구 품질 평가에서 낮고 바깥쪽 코스의 효과와 카운트의 중요성을 확인하였다. 이들 모델은 각 투구를 독립 사건으로 다루므로 배합의 순서 효과를 포착하기 어렵다는 한계가 있다.

이를 보완하기 위해 시퀀스 모델이 도입되었다. Kneita 는 Transformer 기반 투구 결과 예측에서 시퀀스 맥락과 투수 강점 정렬의 중요성을 보고하였다. 초기 버전의 본 연구도 이 계열에 속하며, 긴 시퀀스에서 현재 상황 정보가 희석되는 문제를 지적하고 맥락을 출력층에 직접 결합하는 구조를 제안하였다.

가장 최근의 연구인 Takamido 와 Nakamoto(2026)는 반사실(counterfactual) 분석을 도입하여, 실제로 던지지 않은 공의 가치를 평가하는 틀을 제시하였다. 이들은 타석 단위 시퀀스와 맥락 정보를 별도 경로로 처리하는 이원 구조를 사용하였고, 최종구뿐 아니라 그 직전의 셋업 피치를

대체하며 시즌 지표(K/9, ERA, oSLG)에 미치는 영향을 추정하였다. 본 개정판은 이 연구의 데이터 구성 방식 — 타석 단위 시퀀스, 좌타자 좌표 미러링, 타자별 스트라이크존 정규화, 투수×구종 실측 평균을 이용한 반사실 대체, 제구 오차를 반영한 윈도우 평균 — 을 다수 채택하였다.

특기할 점은 이들이 보고한 결과의 방향이다. 셋업 피치 최적화의 효과(K/9 +1.10~1.23)는 최종구 최적화의 효과(+1.26~3.54)보다 일관되게 작았다. 후술하듯 본 연구의 어블레이션 결과도 같은 방향을 가리킨다.

3. 초기 구현의 재현과 결함 분석

개정 작업은 초기 구현의 재현에서 시작하였다. 학습 노트북과 원본 데이터를 확보하여 전처리 과정을 추적한 결과, 보고된 성능을 무효화하는 결함들이 확인되었다.

3.1 시간 역순 시퀀스로 인한 미래 정보 누수

pybaseball 이 제공하는 Statcast 원본은 역시간순으로 정렬되어 있다. 실제 데이터를 확인한 결과 한 타석 안에서 pitch_number 가 7, 6, 5 ... 1 의 순서로 나타났으며, 전체 629,843 행에서 날짜가 과거로 이동하는 전환이 158 회, 미래로 이동하는 전환이 5 회였다. 초기 코드에는 정렬 구문이 존재하지 않았다.

```
grouped = df.groupby('batter')          # 역순이 그대로 유지된다
sequences.append(data[i : i+8])
targets.append(target_vals[i+7])        # 윈도우의 마지막 = 시간상 가장 과거
```

그 결과 모델은 타깃보다 **나중에** 던져진 7 개의 공을 입력으로 받아 그보다 **먼저** 던져진 1 개 공의 결과를 예측하고 있었다. 더욱이 볼카운트가 모든 타임스텝에 포함되어 있었으므로, 타깃 다음 투구의 카운트를 비교하는 것만으로 타깃의 결과를 직접 읽어낼 수 있었다. 볼이 1 증가했으면 볼, 스트라이크가 1 증가했으면 스트라이크이며, 0-0 으로 초기화되었다면 그 투구가 타석을 종료한 것이다. 전체의 절반 이상을 차지하는 볼과 스트라이크가 입력에서 직접 관측 가능했다.

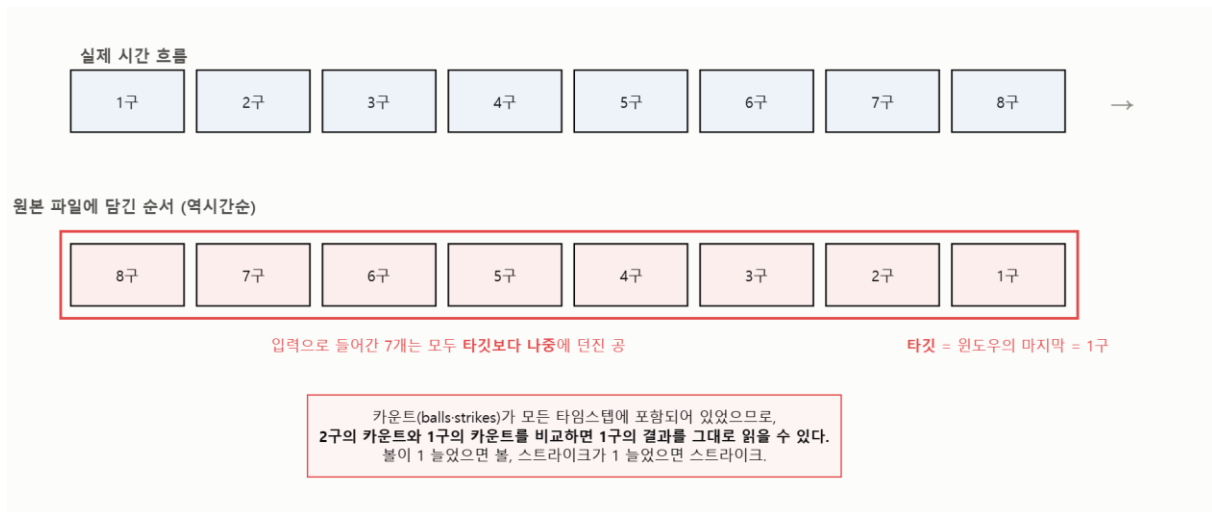


그림 1. 정렬 누락이 만들어 낸 누수 구조. 원본이 역시간순이므로 정렬 없이 잘라낸 윈도우에서 타깃은 시간상 가장 과거의 투구가 되고, 나머지 입력은 모두 그 이후에 일어난 사건이 된다.

3.2 무작위 분할과 겹치는 윈도우

초기 보고서는 '시간 순서에 따라 8:2 로 분할'하였다고 기술하였으나, 실제 코드는 `train_test_split` 에 `stratify` 를 지정한 무작위 셔플 분할이었다. 여기에 `stride 2` 의 슬라이딩 윈도우가 결합되어, 학습 표본 $[i..i+7]$ 과 검증 표본 $[i+2..i+9]$ 가 8 개 중 6 개의 투구를 공유하였다. 검증셋이 학습셋과 사실상 중복된 상태였다.

3.3 불균형 이중 보정에 의한 확률 왜곡

학습은 역빈도 가중 오버샘플링(`WeightedRandomSampler`)과 `Focal Loss` 를 동시에 적용하였다. 불균형을 두 번 보정한 셈이며, 그 결과 모델의 출력 확률은 실제 분포가 아니라 균등화된 분포를 가리키게 된다. 응용 단계에서 `temperature` 를 5.0 으로 고정하여 분포를 인위적으로 평탄화한 것은 이 왜곡을 가리기 위한 대증요법이었다.

3.4 결과 정의와 데이터 범위의 문제

- 정규시즌 필터가 없어 시범경기 35,482 구가 학습에 포함되어 있었다.

- 결과 매핑이 events 화이트리스트에 의존하여 3 루타, 희생플라이, 병살, 야수선택 등 **인플레이 결과 10,224 구가 삭제**되었다. 3 루타가 클래스에서 사라진 것도 이 때문이다.
- 수집 범위가 2024 년 3~8 월에 그쳤음에도 '정규 시즌 전체'로 기술되었다.
- 인코더와 MinMaxScaler 를 **전체 데이터에 적합**한 뒤 분할하여, 검증셋의 분포 정보가 정규화를 통해 학습에 유입되었다.

3.5 추천 점수의 수학적 무의미성

초기 추천 점수는 다음과 같이 정의되어 있었다.

$$\text{score} = p_{\text{success}} - (p_{\text{fail}} * 1.5) - \text{penalty}$$

그러나 good 집합(5 개)과 bad 집합(6 개)이 11 개 클래스 전체를 정확히 이분하므로 항상 $p_{\text{fail}} = 1 - p_{\text{success}}$ 가 성립한다. 따라서 $\text{score} = 2.5 \cdot p_{\text{success}} - 1.5$ 이며, **위험 가중치 1.5 는 후보 순위에 어떠한 영향도 주지 못한다.** 또한 2 스트라이크에서의 파울과 범타가 동일하게 '성공'으로 분류되어, 실제 가치 차이가 반영되지 않았다.

4. 제안 방법

3장에서 확인한 결함에 대응하여 데이터 구성, 모델, 평가, 추천 로직을 재설계하였다. 설계 원칙은 세 가지다. 첫째, 학습 시점에 알 수 없는 정보는 어떤 경로로도 입력에 들어오지 않는다. 둘째, 성능 수치는 학습·조기종료·모델선택에 한 번도 쓰이지 않은 구간에서 단 한 번 측정한다. 셋째, 확률로 후보를 순위화하는 시스템이므로 확률의 정직성(캘리브레이션)을 정확도보다 우선한다.

4.1 데이터 구성

2024-2025 정규시즌 전체를 수집하여 총 1,424,426 구를 확보하였다(2024 년 711,898 구, 2025 년 712,528 구). game_type 이 정규시즌인 경기만 사용하였다. 원본은 game_date → game_pk → at_bat_number → pitch_number 순으로 정렬하여 시간축을 복원하였다.

시퀀스는 타석(plate appearance) 내부로 제한한다. 각 투구를 타깃으로 삼고, 같은 타석에서 그보다 앞서 던져진 공들을 최대 6 구까지 앞에 붙인다. 부족한 앞부분은 0 으로 채우고 마스크로 표시하여 어텐션에서 제외한다. 타석 경계를 넘지 않으므로 다른 타자나 투수의 투구가 한 시퀀스에 섞이지 않는다. 실제 데이터에서 타석당 평균 투구 수는 3.82 구이고 91.7%의 타석이 6 구 이하이므로, 길이 6 은 대부분의 타석을 온전히 포괄한다.

좌표계는 Takamido 와 Nakamoto(2026)를 따라 정규화하였다. 좌타자의 좌우 위치와 수평 무브먼트는 부호를 반전시켜 좌우 타자를 동일한 의미 축에서 비교할 수 있게 하고, 높이는 타자별 스트라이크존 상·하단으로 정규화하여 신장 차이를 흡수한다. 회전축은 각도이므로 사인·코사인으로 분해하여 0 도와 359 도가 인접하도록 하였다.

구분	변수	개수
시퀀스 (투구 물리량)	좌우 위치, 정규화 높이, 구속, 회전수, 회전축(sin·cos), 수평·수직 무브먼트	8
시퀀스 (범주형)	타자, 투수, 구종, 타석 좌우, 투구 좌우	5

맥락	볼, 스트라이크, 아웃, 주자 1·2·3 루, 이닝, 초말, 점수차, 타순 회차	10
맥락 (타자 특성)	직전 시즌 삼진율, 홈런율, ISO, 타율, OPS	5

표 1. 입력 변수 구성. 맥락의 타자 특성은 **직전 시즌** 성적만 사용하여 같은 시즌 정보가 유입되지 않도록 하였다.

볼카운트를 시퀀스 경로에서 제외한 것은 의도적인 설계다. 3.1 에서 확인한 누수는 카운트가 모든 타임스텝에 존재했기 때문에 발생하였다. 카운트를 맥락 경로에만 두면 타깃 시점의 상황은 그대로 활용하면서, 인접 투구의 카운트를 비교해 정답을 추론하는 경로 자체가 구조적으로 차단된다.

4.2 출력 클래스의 재정의

초기 정의는 strike 와 strikeout 을 동시에 클래스로 두었다. 그러나 삼진은 2 스트라이크에서의 스트라이크이고 볼넷은 3 볼에서의 볼이므로, 이들은 투구의 결과가 아니라 **카운트에서 파생되는 타석 결과**다. 카운트가 입력에 포함된 상태에서 이를 별도 클래스로 두면 모델이 사실상 공짜로 맞히는 라벨이 생긴다. 본 개정에서는 투구 단위로 실제 관측되는 사건만을 클래스로 정의하였다.

```
ball · called_strike · swinging_strike · foul · hit_by_pitch
field_out · single · double · triple · home_run
```

삼진과 볼넷은 예측된 클래스와 현재 카운트로부터 결정적으로 유도된다. 또한 인플레이 결과 매핑을 재정의하여 병살, 희생플라이, 야수선택, 실책 출루 등을 포함시켰다. 그 결과 매핑되지 않아 삭제되는 투구가 10,224 구에서 6 구로 감소하였다.

4.3 모델 구조

제안 모델은 시퀀스와 맥락을 별도 경로로 처리한 뒤 출력 직전에 결합하는 이원 구조를 갖는다. 이는 초기 버전이 제안한 구조를 계승한 것이며, Takamido 와 Nakamoto(2026)가 독립적으로 채택한 구조와도 일치한다.

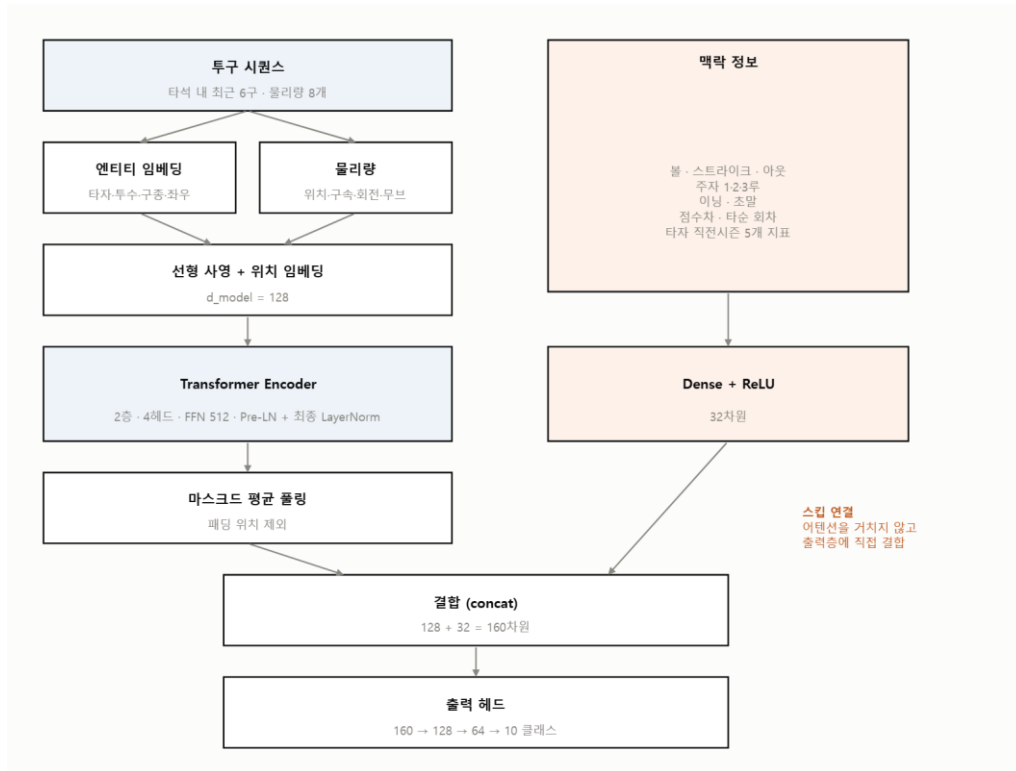


그림 2. 제안 모델 구조.

각 구성요소의 역할과 설계 근거는 다음과 같다.

(1) 엔티티 임베딩

타자·투수·구종 등 고차원 범주형 변수를 학습 가능한 밀집 벡터로 사상한다. 원-핫 인코딩과 달리 선수 간 유사성이 벡터 공간에 표현될 수 있다. 학습 데이터에 등장한 타자 624 명, 투수 802 명이 사전에 포함된다. **인덱스 0 은 UNK(미등록 선수)로 예약**하고, 학습 중 일정 확률로 선수 ID 를 UNK 로 치환하여 해당 임베딩이 실제로 훈련되도록 하였다. 초기 구현은 예외 처리로 미등록 선수를 0 번 선수로 조용히 대체하여, 처음 보는 선수에 대해 특정 선수의 예측을 반환하는 오류가 있었다.

(2) Transformer 인코더

d_model 128, 헤드 4 개, 레이어 2 층, 피드포워드 차원 512 로 구성하였다. 셀프 어텐션이 타석 내 투구 간 관계를 학습한다. 세 가지를 수정하였다.

- **최종 LayerNorm 추가.** norm_first=True(Pre-LN) 설정에서는 마지막 블록의 출력이 정규화되지 않은 채로 헤드에 전달된다. 초기 구현에는 최종 정규화가 없어 잔차 스트림의 크기가 제어되지 않았다.
- **패딩 마스크 적용.** 타석 앞부분의 패딩 위치에도 어텐션이 걸리고 있었다. src_key_padding_mask 로 존재하지 않는 투구를 제외하였다.
- **마스크드 평균 풀링.** 마지막 토큰만 사용하던 것을 유효 구간 평균으로 변경하였다. 짧은 타석에서 특히 안정적이다.

(3) 맥락 경로와 스킵 연결

맥락 정보는 Transformer 를 거치지 않고 별도의 완전연결층을 통과한 뒤 출력 직전에 결합된다. 이것이 초기 연구가 제안한 핵심 구조다. 다만 초기 구현에는 심각한 문제가 있었다. 원시 정수 6 개를 Linear(6, 32)와 ReLU 에 통과시켰는데, **0-0 카운트·무사·주자 없음이면 입력 벡터가 전부 0 이 되어 ReLU(bias)라는 상수만 남는다.** 즉 가장 흔한 상황에서 스킵 연결이 전달하는 정보량이 0 이었다. 본 개정에서는 맥락 피처를 표준화하여 0 이 특별한 값이 되지 않게 하고, 이닝·점수차·타순 회차·타자 직전시즌 성적을 추가하여 표현력을 높였다.

(4) 출력 헤드와 보조 회귀

결합된 벡터는 다층 퍼셉트론을 거쳐 10 개 클래스의 로짓을 산출한다. 정규화는 BatchNorm 대신 LayerNorm 을 사용하였다. BatchNorm 은 배치 구성에 민감하여 추론 시 배치 크기가 달라지는 응용 환경에 적합하지 않기 때문이다. 추가로 해당 투구의 기대 득점 변화량을 직접 예측하는 보조 회귀 헤드를 두어, 분류 손실과 함께 학습하였다.

최종 모델의 파라미터 수는 486,199 개(약 2.0MB)이다. 초기 모델(5,408,636 개, 약 21MB)의 약 1/11 규모다. 초기 보고서는 '경량화 달성'을 측정 없이 주장하였으나, 본 개정에서는 실측값으로 제시한다.

4.4 학습 설정

2024 시즌을 학습에, 2025 시즌을 테스트에 사용한다. 검증셋은 학습 구간의 **뒷부분 날짜**에서 15%를 분리하였다. 무작위 분할과 달리 같은 타석의 이웃 투구가 학습과 검증으로 갈리지 않는다. 인코더와 정규화 통계는 학습 구간에만 적합하고 그 값을 검증·테스트에 그대로 적용한다.

손실 함수는 단순 교차 엔트로피를 사용하고 **오버샘플링을 적용하지 않았다**. 3.3 에서 확인하였듯 이중 보정은 확률을 왜곡한다. 희귀 클래스의 재현율을 높이는 대신 확률의 정직성을 택한 것이며, 이 선택의 결과는 5.4 에서 정량적으로 검토한다. 최적화는 AdamW(학습률 3×10^{-4} , weight decay 10^{-4}), 스케줄러는 ReduceLROnPlateau, 그래디언트 클리핑 1.0 을 적용하였다. **조기 종료 기준은 검증 log-loss**이며, 정확도의 우연한 최고점을 성능으로 채택하지 않는다.

4.5 추천 로직

추천은 후보 생성과 점수화의 두 단계로 이루어진다. 후보는 해당 투수가 실제로 던지는 구종과 9×9 좌표 격자의 조합으로 생성한다. 초기 구현은 존 안 9 칸만을 후보로 삼아 **유인구를 추천할 수 없었으나**, 본 개정은 격자를 존 밖까지 확장하였다. 각 구종의 물리량은 리그 평균 상수가 아니라 **해당 투수의 실측 평균**을 사용한다. 현재 참조 테이블은 투수 1,010 명에 대해 3,950 개의 (투수, 구종) 조합을 보유한다.

점수는 모델이 산출한 결과 확률 분포를 카운트 조건부 기대 실점으로 가중합한 값이다.

$$\text{score}(p) = \sum_c P(\text{결과} = c \mid \text{투수 } p, \text{ 상황}) \times \Delta \text{RE}(c \mid \text{볼카운트})$$

여기서 ΔRE 는 Statcast 가 제공하는 투구별 기대 득점 변화량(delta_run_exp)에서 직접 추정한다. 표 2 에서 보듯 같은 '볼'이라도 0-0 에서는 거의 무해하지만 3-2 에서는 볼넷이 되어 8 배 이상의 값을 갖는다. 2 스트라이크에서의 파울은 거의 무가치한 반면 범타는 큰 이득이다. 초기 점수 함수의 이분법은 이러한 차이를 전혀 담지 못하였다.

결과	0-0	0-2	1-2	2-2	3-2
볼	+0.036	+0.020	+0.039	+0.096	+0.287
루킹 스트라이크	-0.040	-0.169	-0.189	-0.228	-0.325

헛스윙 스트라이크	-0.041	-0.168	-0.188	-0.228	-0.323
파울	-0.040	-0.010	-0.010	-0.013	-0.017
인플레이 아웃	-0.253	-0.167	-0.187	-0.225	-0.320
단타	+0.423	+0.536	+0.519	+0.481	+0.383
홈런	+1.522	+1.628	+1.610	+1.557	+1.498

표 2. 카운트 조건부 기대 실점(투수 관점, 음수가 유리). 2024-2025 시즌 `delta_run_exp` 에서 추정하였다.

마지막으로 제구 오차를 반영한다. 투수는 한 점을 정확히 겨눌 수 없으므로, 노린 지점 주변 윈도우의 평균 기대 실점을 그 지점의 대표값으로 삼는다(Takamido 와 Nakamoto 의 `command window`). 윈도우 크기를 조절하면 제구가 정밀한 투수와 그렇지 않은 투수를 구분하여 평가할 수 있다.

5. 실험 결과

5.1 학습 경과

학습은 40 에폭 진행되었으며 검증 log-loss 는 1.1913 에서 1.1120 까지 감소하였다. 최저값이 마지막 에폭인 40 에폭에서 관측된 점에 주목할 필요가 있다. **조기 종료**가 아니라 **에폭 상한에 도달하여 학습이 종료된 것**이므로, 현재 성능은 수렴값이 아니라 주어진 예산 안에서의 값이다. 더 긴 학습으로 개선의 여지가 남아 있다.

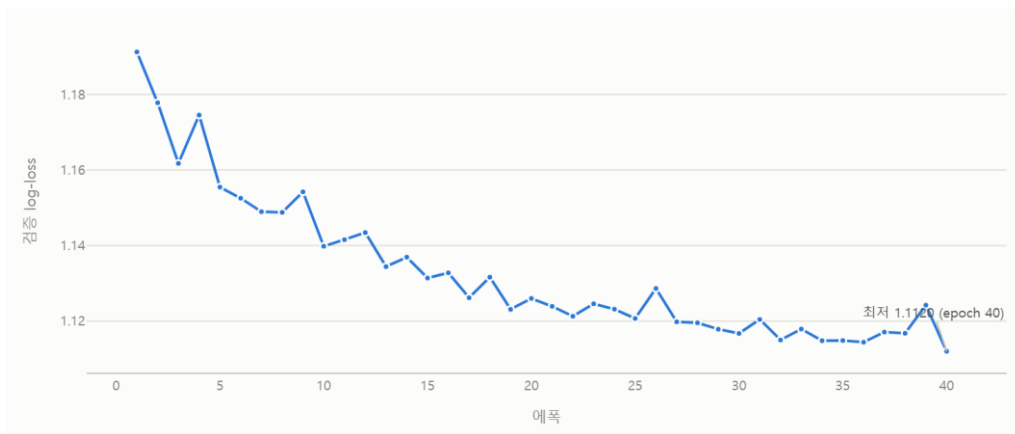


그림 3. 검증 log-loss 의 에폭별 추이.

5.2 테스트 시즌 성능

학습·조기종료·모델선택에 한 번도 사용하지 않은 2025 시즌 712,516 구에서 다음 성능을 얻었다.

지표	값	비고
log-loss	1.1259	주 지표. 낮을수록 좋음
macro AUC	0.8410	클래스 균등 가중 순위 성능
macro F1	0.2633	희귀 클래스 반영 (5.4 참조)
정확도	57.48%	참고 지표
ECE	0.0084	캘리브레이션 오차 (5.3 참조)

표 3. 2025 시즌 홀드아웃 성능.

초기 보고서의 67.52%와 본 결과의 57.48%는 **비교 대상이 아니다**. 전자는 미래 정보 누수를 포함하고, 카운트에서 파생되는 `strikeout·walk` 를 클래스로 포함하며, 시범경기가 섞여 있고, 검증셋이 학습셋과 6/8 을 공유한 상태에서 측정되었다. 동일한 지표명을 공유할 뿐 측정 대상이 서로 다르다.

5.3 캘리브레이션

예측 확률의 정직성을 확인하기 위해 신뢰도 보정 오차(ECE)를 측정하고, 검증셋에서 temperature scaling 계수를 적합하였다. ECE 는 0.0084, 적합된 temperature 는 1.070 였다. **T 가 1 에 근접한다는 것은 모델이 이미 보정되어 있어 추가 보정이 불필요함을 뜻한다**. 실제로 보정을 적용해도 log-loss 는 1.1259 에서 1.1248 로 거의 변하지 않았다.

이는 4.4 의 설계 선택에 대한 직접적인 근거다. 초기 구현이 temperature 5.0 을 손으로 고정해야 했던 것은 이중 보정으로 왜곡된 확률을 가리기 위함이었고, 원인을 제거하자 보정 자체가 필요 없어졌다. 화면에 표시되는 확률이 실제 빈도와 일치하므로, 추천 점수의 기대 실점 계산도 신뢰할 수 있다.

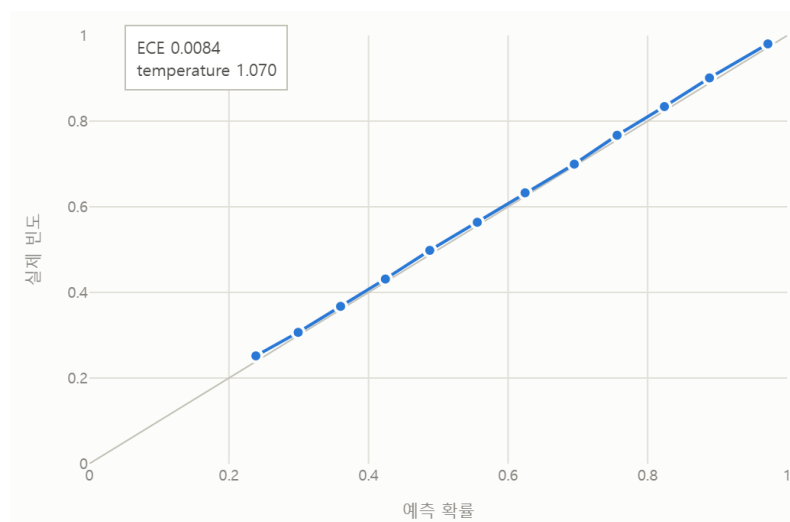


그림 4. 캘리브레이션 곡선. 대각선에 가까울수록 예측 확률이 실제 빈도와 일치한다.

5.4 클래스별 성능과 설계상의 맞바꿈

클래스	표본	정밀도	재현율	F1	AUC
볼	255,753	0.788	0.877	0.830	0.951
루킹 스트라이크	115,670	0.525	0.704	0.601	0.900
헛스윙 스트라이크	78,324	0.361	0.182	0.242	0.777
파울	135,954	0.400	0.510	0.448	0.789
사구	1,928	0.563	0.146	0.232	0.994
인플레이 아웃	83,352	0.336	0.239	0.280	0.798
단타	27,512	0.000	0.000	0.000	0.785
2 루타	7,745	0.000	0.000	0.000	0.791
3 루타	628	0.000	0.000	0.000	0.791
홈런	5,650	0.000	0.000	0.000	0.835

표 4. 2025 시즌 클래스별 성능.

안타 계열(단타·2 루타·3 루타·홈런)의 재현율이 0 이다. 사전 확률이 1~4%인 클래스는 argmax 에서 최상위가 되지 못하기 때문이며, macro F1 이 낮은 것도 대부분 이 때문이다. 그러나 같은 클래스들의 **AUC 는 0.785~0.835 로, 순위 매기기는 정상적으로 이루어지고 있다.** 즉 모델은 '어느 투구가 장타로 이어질 가능성이 높은가'를 구분하지만, 그 확률이 다른 클래스를 넘어설 만큼 크지는 않다.

이는 의도한 맞바꿈이다. 재현율을 높이려면 오버샘플링이나 클래스 가중을 도입해야 하는데, 그러면 출력 확률이 실제 분포를 벗어나 캘리브레이션이 훼손된다(초기 구현이 정확히 그 상태였다). 본 시스템의 추천 엔진은 argmax 가 아니라 **확률 분포 전체를 기대 실점으로 적분**하므로, 여기서는 정직한 확률이 높은 재현율보다 중요하다.

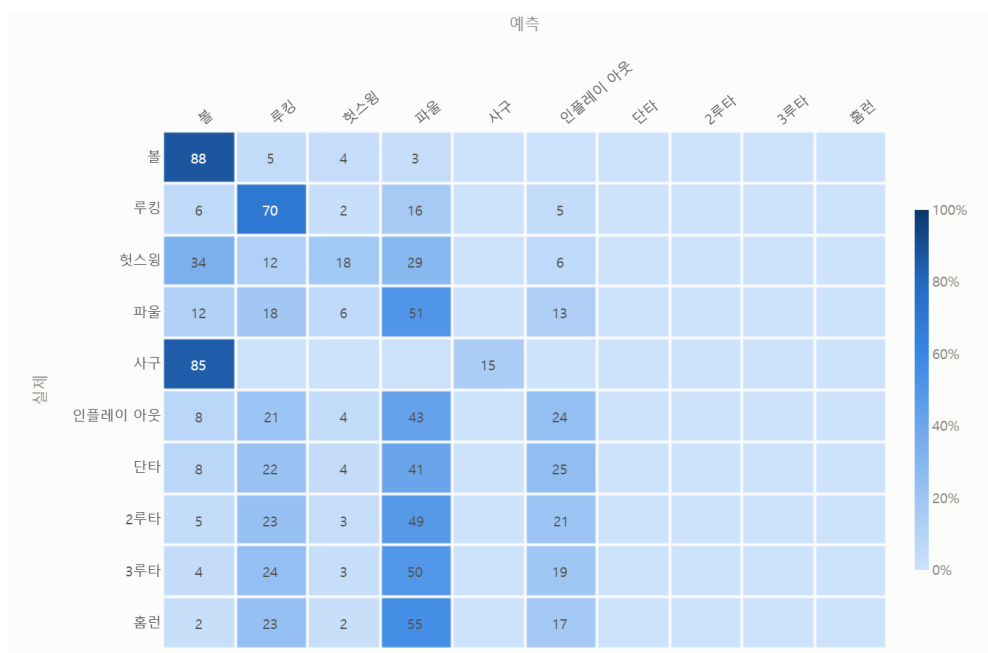


그림 5. 혼동 행렬(행 정규화, 단위 %).

5.5 어블레이션

초기 보고서의 가장 큰 방법론적 약점은 어블레이션의 부재였다. 제안 모델과 베이스라인이 d_model, 레이어 수, 손실 함수, 에폭 수, 학습률까지 모두 달랐으므로 성능 차이를 특정 요소에 귀속시킬 수 없었다. 본 개정에서는 **한 번에 하나의 요소만 변경**하고 나머지 조건을 동일하게 유지하였다.

구성	log-loss	제안 대비	macro AUC	정확도(%)	macro F1
상태 임베딩 제거	1.1244	-0.0014	0.8412	57.53	0.2573
시퀀스 길이 10	1.1254	-0.0005	0.8407	57.46	0.2568
제안 모델 (전체)	1.1259	+0.0000	0.8410	57.48	0.2633
마지막 토큰 풀링	1.1269	+0.0010	0.8400	57.42	0.2683
시퀀스 길이 3	1.1288	+0.0029	0.8407	57.29	0.2476
시퀀스 길이 1	1.1302	+0.0043	0.8393	57.36	0.2627
맥락 스킵 연결 제거	1.1423	+0.0165	0.8364	56.91	0.2571

표 5. 어블레이션 결과(2025 시즌 홀드아웃). 굵은 행이 제안 모델이다.

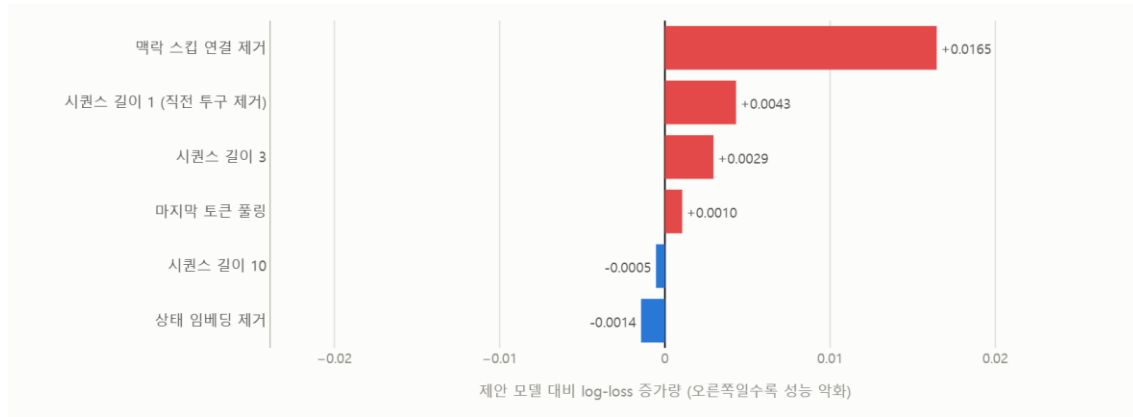


그림 6. 제안 모델 대비 log-loss 증가량. 오른쪽으로 길수록 해당 요소를 제거했을 때 성능이 크게 나빠진다.

(1) 맥락 스킵 연결은 실제로 기여한다.

제거 시 log-loss 가 +0.0165 증가하고 macro AUC 는 -0.0046 감소하였다. 표에서 가장 큰 효과이며, 유일하게 노이즈 범위 밖으로 판단되는 차이이다. 초기 연구가 제안한 핵심 구조가 공정한 비교 하에서 지지된 것이다.

(2) 시퀀스 자체의 기여는 작다.

시퀀스 길이를 1 로 두어 직전 투구 정보를 완전히 제거해도 log-loss 차이는 +0.0043 에 그쳤다. 이는 맥락 스킵 연결 효과의 약 26% 수준이다. 예측력의 대부분은 해당 투구 자체의 물리량과 현재 상황·타자 특성에서 나오며, 직전 투구들이 추가로 제공하는 정보는 제한적이다. **'투구 배합의 시계열 맥락이 결정적'**이라는 초기 연구의 전제는 이 데이터에서 강하게 지지되지 않는다. 셋업 피치 최적화의 효과가 최종구 최적화보다 작았던 Takamido 와 Nakamoto(2026)의 보고와도 방향이 일치한다.

(3) 카운트·베이스아웃 상태 임베딩은 효과가 없다.

제거해도 log-loss 가 -0.0014 로 오히려 미세하게 개선되었다. 맥락 경로의 이득은 이산 상태 임베딩이 아니라 연속 맥락 피쳐, 특히 타자의 직전 시즌 성적에서 나오는 것으로 해석된다. 이 구성요소는 제거하는 것이 타당하다.

6. 시스템 구현

모델과 분석 결과를 현장에서 활용할 수 있도록 Streamlit 기반 웹 대시보드를 구현하였다. 다섯 개 화면으로 구성되며, 학습 산출물이 없어도 데이터 집계 기반 화면은 독립적으로 동작한다.

화면	기능
투구 추천	매치업·상황·타석 이력을 입력하면 구종×코스 후보 전체를 평가하여 기대 실점이 가장 낮은 상위 3 개를 제시. 구종별 코스 지도 동시 표시
타자 분석	코스별·구종별 약점, 헛스윙률·체이스율 등 지표를 리그 평균 대비로 표시
투수 분석	구종 레퍼토리와 무브먼트, 카운트별 구종 선택 성향, 코스 성향
팀 분석	30 개 구단 투수진 지표 순위와 소속 선수 명단
모델 성능	테스트 시즌 지표, 학습 곡선, 캘리브레이션, 혼동 행렬, 어블레이션

표 6. 대시보드 구성.

선수 프로파일은 2024·2025 전체 데이터로 산출하였으며, 분석 가능 대상은 타자 1,024 명, 투수 1,193 명이다. 코스별 지표는 표본이 적은 칸에서 우연한 극단값이 약점으로 오인되지 않도록 경험적 베이지 축소를 적용하여 리그 평균 쪽으로 당겼다.

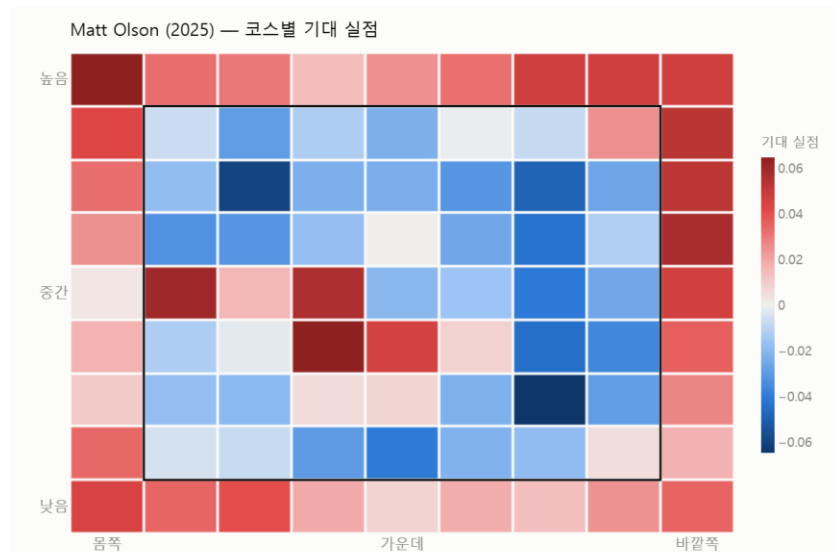


그림 7. 타자 코스별 기대 실점 예시. 빨강은 타자에게 유리한(투수에게 위험한) 구역, 파랑은 그 반대다. 검은 사각형이 스트라이크존이다.

시각화는 전 화면에서 일관된 규약을 따른다. 구종별 색은 구종에 고정 배치되어 필터를 변경해도 바뀌지 않으며, 부호가 의미를 갖는 값은 발산형(파랑-회색-빨강), 크기만을 나타내는 값은 단일 색조 순차형으로 표현한다. 사용한 범주형 팔레트는 명도대·채도·색각 분리 기준에 대해 사전 검증하였고, 모든 차트에 직접 라벨과 표 형태의 대체 표현을 함께 제공하여 색만으로 값을 읽지 않아도 되게 하였다.

7. 논의 및 한계

본 연구의 한계와 후속 과제는 다음과 같다.

- **단일 시드 실행.** 표 5의 어블레이션은 시드 1개의 단일 실행 결과다. 맥락 스킵 연결의 차이(log-loss +0.0164)는 노이즈 범위 밖으로 보이나, 나머지 구성의 차이(0.001~0.004)는 재현 실험 없이 결론짓기 어렵다. 핵심 3개 구성에 대해 시드를 늘려 평균과 표준편차로 보고할 필요가 있다.
- **학습 예산의 미포화.** 5.1에서 지적하였듯 검증 log-loss의 최저값이 마지막 에폭에서 관측되었다. 더 긴 학습에서 성능이 개선될 여지가 있으며, 현재 수치는 하한으로 보아야 한다.
- **희귀 클래스 재현율.** 안타 계열의 재현율이 0인 것은 캘리브레이션을 우선한 설계의 결과다. 확률 보정을 유지하면서 희귀 사건 탐지를 개선하는 방법(사후 임계값 조정, logit adjustment 등)의 검토가 필요하다.
- **추천기 자체의 검증 부재.** 본 연구는 결과 예측 모델을 홀드아웃에서 평가하였으나, 추천 로직이 실제로 더 나은 선택을 하는지는 검증하지 않았다. Takamido와 Nakamoto(2026)의 반사실 최적화와 시즌 지표 회귀를 적용하면 '이 추천을 따랐을 때 K/9가 얼마나 개선되는가'를 정량화할 수 있다. 본 개정에서 참조 테이블과 반사실 후보 생성 기반은 이미 구축하였으므로, 이 분석은 자연스러운 후속 과제다.
- **타자 적응의 미고려.** 최적 선택이 특정 구종·코스로 수렴하면 타자의 예측 가능성이 높아져 실제 효과가 추정치보다 낮아질 수 있다. 투수와 타자의 전략적 상호작용을 모델에 반영하는 것이 필요하다.

한편 본 개정 과정에서 얻은 방법론적 교훈을 기록해 둔다. **성능이 예상보다 높게 나올 때 그것을 검증하는 절차가 파이프라인에 포함되어야 한다.** 초기 구현의 67.52%는 그 자체로는 의심스럽지 않은 값이었고, 정확도라는 단일 지표로는 누수를 감지할 수 없었다. 본 개정에서는 시퀀스 조립 후 '유효 길이가 $\min(\text{pitch_number}, 6)$ 과 일치하는가'를 자동 검증하는 스크립트를 파이프라인에 포함시켰다. 이러한 구조적 점검이 사후 분석보다 효과적이다.

8. 결론

본 연구는 초기 구현이 보고한 성능이 미래 정보 누수에 기인함을 규명하고, 데이터 구성부터 평가 방법까지 파이프라인을 재설계하였다. Statcast 원본의 역시간순 정렬을 놓친 것이 근본 원인이었으며, 볼카운트가 모든 타임스텝에 포함되어 있어 인접 투구의 카운트 비교만으로 정답을 읽을 수 있는 구조였다.

재설계 후 2025 시즌 홀드아웃에서 log-loss 1.1259, macro AUC 0.8410, ECE 0.0084 를 얻었다. 정확도는 초기 보고값보다 낮으나, 두 수치는 측정 대상이 다르므로 비교 가능한 값이 아니다. 오히려 temperature 가 1.070 로 1 에 근접한다는 사실은 출력 확률이 실제 빈도와 일치함을 보여주며, 확률로 후보를 순위화하는 본 시스템에서는 이것이 정확도보다 본질적인 성질이다.

어블레이션을 통해 맥락 스킵 연결의 기여를 공정한 조건에서 확인하였고, 동시에 투구 시퀀스의 기여가 예상보다 작다는 부정적 결과도 함께 얻었다. 후자는 초기 연구의 전제를 약화시키지만, 그 자체로 의미 있는 발견이다. 투구 배합의 효과를 정량화하려면 본 연구보다 정교한 실험 설계가 필요하며, 이는 반사실 분석과 시즌 지표 환산을 결합한 후속 연구로 이어질 수 있다.

참고문헌

- [1] R. Takamido and H. Nakamoto, "Counterfactual Optimization of Baseball Pitch Sequences and Estimation of Its Impact on Season-Level Statistics," arXiv:2606.17345, 2026.

- [2] D. Kneita, "Transformer-Based Baseball Modeling for Pitch Outcome Prediction and Strategy Optimization," MIT Sloan Sports Analytics Conference, 2025.
- [3] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On Calibration of Modern Neural Networks," in Proc. ICML, 2017, pp. 1321–1330.
- [4] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal Loss for Dense Object Detection," in Proc. IEEE ICCV, 2017, pp. 2980–2988.
- [5] C. Guo and F. Berkhahn, "Entity Embeddings of Categorical Variables," arXiv:1604.06737, 2016.
- [6] A. Vaswani et al., "Attention Is All You Need," in Advances in Neural Information Processing Systems 30, 2017.
- [7] P. Swartz, M. Grosskopf, D. R. Bingham et al., "The Quality of Pitches in Major League Baseball," The American Statistician, vol. 71, no. 2, pp. 148–154, 2017.
- [8] G. Sidle and H. Tran, "Using Multi-Class Classification Methods to Predict Baseball Pitch Types," Journal of Sports Analytics, vol. 4, no. 1, pp. 85–93, 2018.
- [9] A. Prasad, "Decoding MLB Pitch Sequencing Strategies via Directed Graph Embeddings," MIT Sloan Sports Analytics Conference, 2021.
- [10] MLB Advanced Media, "Statcast Data," Baseball Savant. [Online]. Available: <https://baseballsavant.mlb.com>

부록. 재현 방법

본 연구의 코드와 산출물은 아래 저장소에 공개되어 있다. 데이터 수집부터 학습, 산출물 생성까지 세 개의 노트북으로 재현할 수 있다.

https://github.com/hwangiljun/deeplearning_project

단계	실행	내용
1	notebooks/01_download_statcast.ipynb	2024-2025 정규시즌 수집
2	notebooks/02_train.ipynb	학습 및 어블레이션 (GPU)
3	notebooks/03_build_artifacts.ipynb	참조 테이블·프로파일·평가
-	scripts/validate_pipeline.py	누수 차단 자동 검증
-	streamlit run app.py	대시보드 실행